GetX bridge go_router

Bridging the Gap

Incremental Navigation & UI Migration in Flutter Apps

Mateusz Rus

Fluttercon · 7 October 2026

No feature freeze - so we migrate on two tracks

Hybrid Architecture = the old and the new architecture in one Flutter app, both in production.

WHY MIGRATE

- Global state everywhere
- Hidden dependencies
- Hard to test in isolation
- No clear feature boundaries

FEATURES
never stop

New features in the new architecture:
new rules · tests · clear business boundaries

PLATFORM
step by step

Shared foundations + old parts move over,
one at a time (strangler fig)

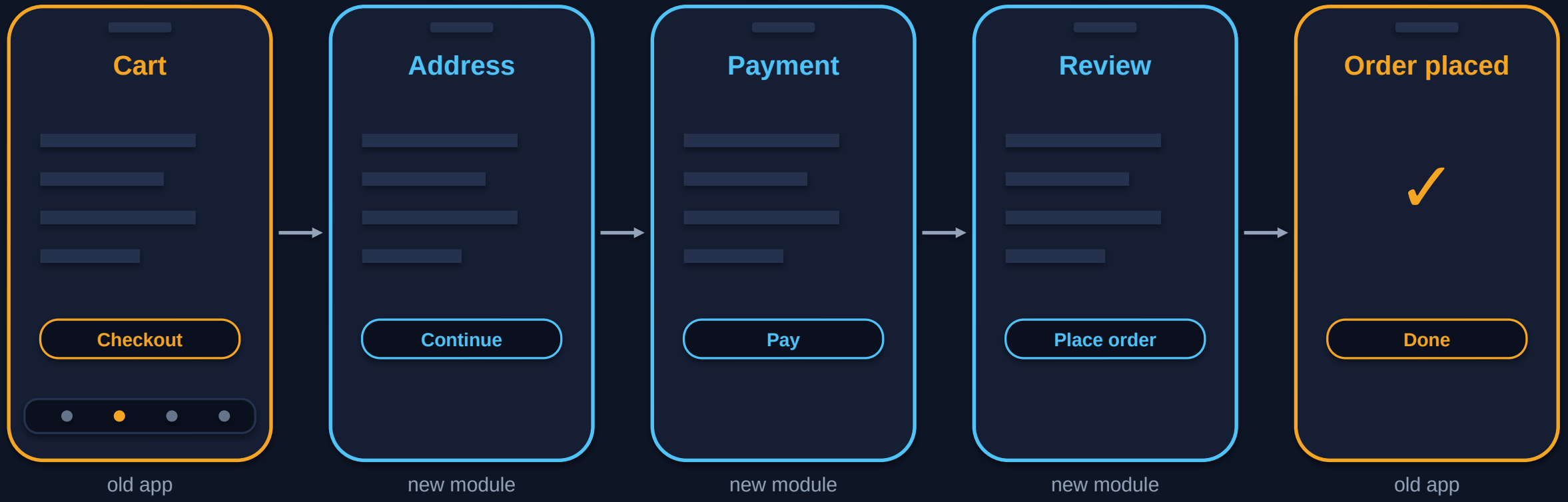
Legacy | **Hybrid Architecture - you live here the longest. Design it on purpose.** | **Target**

Today: **navigation (Navigator 1.0 → 2.0)** · **ports & adapters** · shorter: design system & accessibility

THE EXAMPLE

One journey, old and new code

Checkout: the middle is new. The start and the end are old.

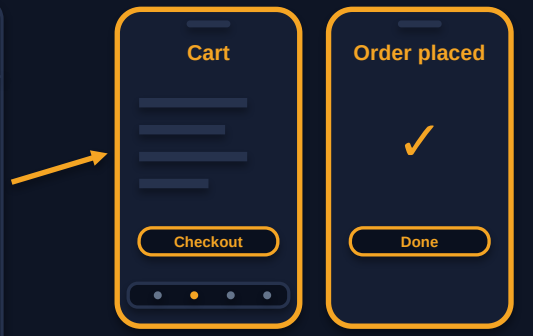


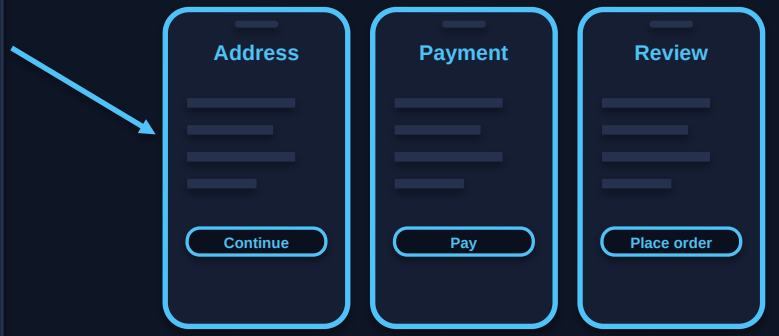
■ old code - the old app (Navigator 1.0) ■ new code - a module (go_router + Bloc)

Where the code lives

tree

```
my_app/                              one repo · pub workspace · melos
├── lib/app/                         old app · GetX · Navigator 1.0
│   ├── modules/cart/                Cart (old)
│   ├── modules/orders/              Order placed (old)
│   └── adapters/                    old meets new - only here
└── packages/
    ├── modules/                     new features · go_router + Bloc
    │   ├── checkout/                Address · Payment · Review
    │   │   ├── lib/checkout.dart    public: handle + ports
    │   │   ├── lib/src/             private: routes · Cubits · data
    │   │   └── test/architecture/   import rules
    │   ├── auth/
    │   └── <feature>/               every new feature, same shape
    └── common/                      shared · clean · no extra deps
        └── core/  network/  routing/  design_system/
```

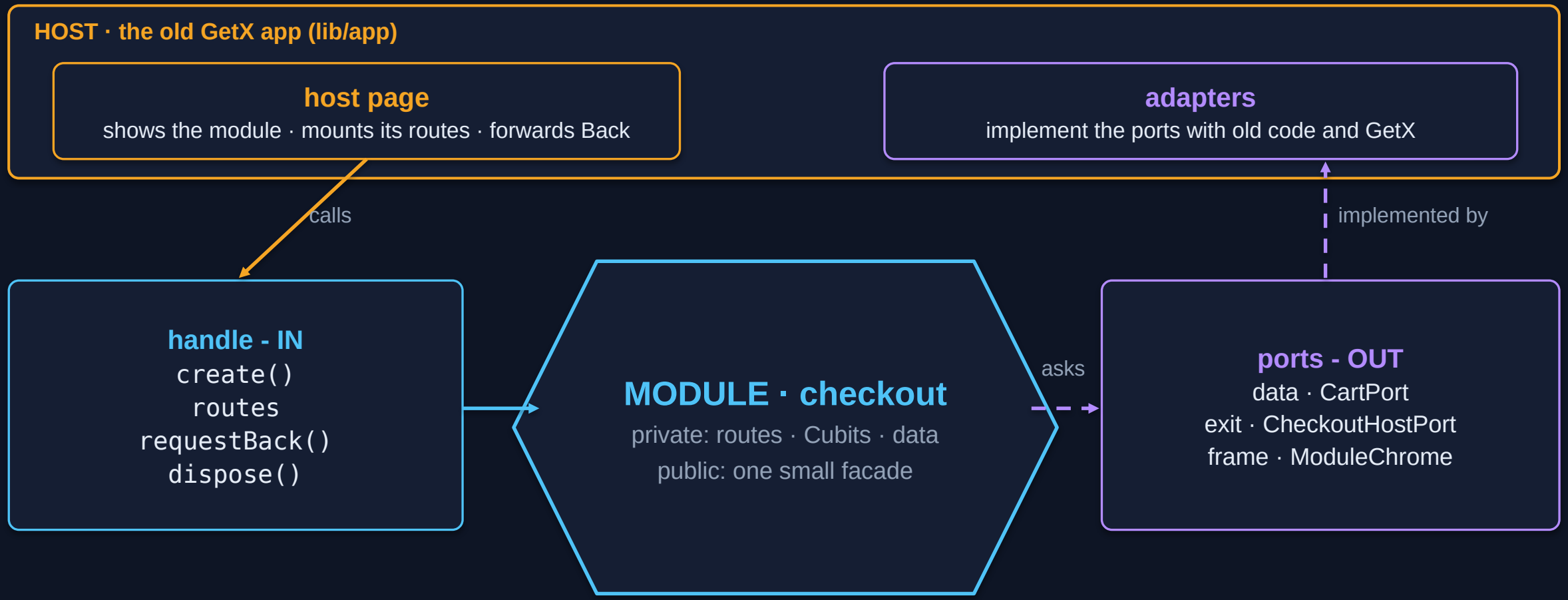




lib/app keeps the old screens.
modules/checkout has the new ones.

ARCHITECTURE

Four words: host · module · port · adapter



Imports point in. Calls go out.

ARCHITECTURE

One rule: imports point in, calls go out

HOST
the old app
lib/app

old code + GetX
live here

```
import 'package:checkout/checkout.dart';
```

✓ the host imports the module

```
import 'package:my_app/…';   import 'package:get/…';
```

× the module never imports the host - or GetX

```
await cart.placeOrder(draft);   // CartPort
```

calls go out: the port → the host's adapter → old code

MODULE
packages/modules/
checkout

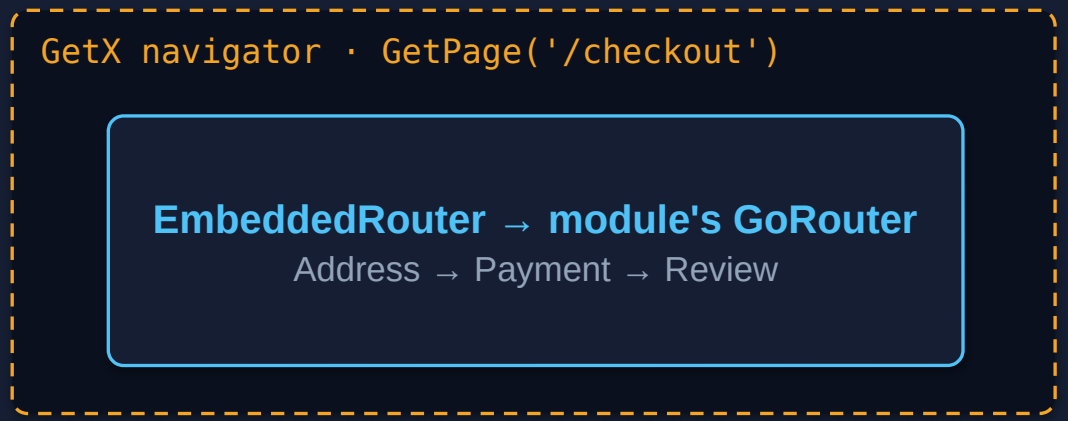
no GetX · no app code

Imports point in. Calls go out. So old code and GetX stay in the host.

Two hosts, one module

Old host: GetX here - or any Navigator 1.0 app. New host: the Router API (go_router).

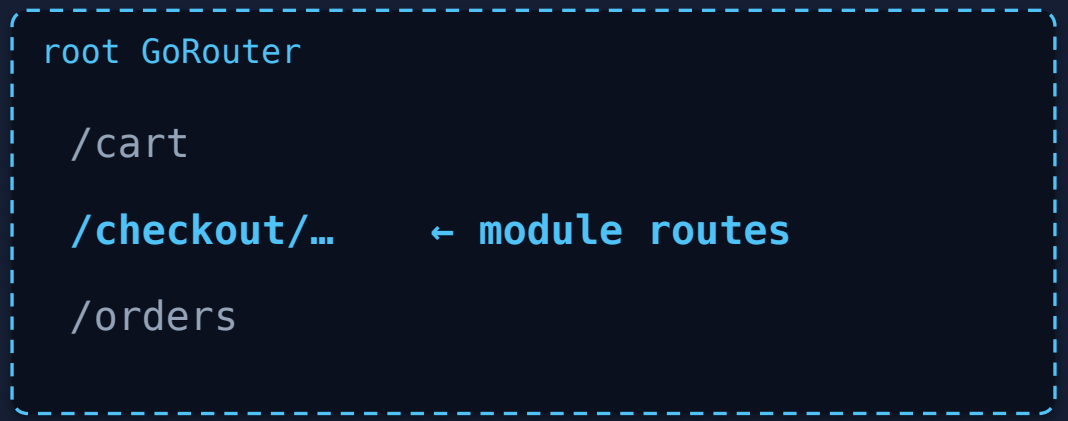
TODAY · old app (GetX) - mobile and web



EMBED

One old page shows the module and its router.

TARGET · new app (go_router)



COMPOSE

Module routes join the root: real URLs, deep links, browser back.

Same module, same routes + guards. One router per navigation tree - only the root router writes the URL.

The router lives and dies with its host page

lib/app - how we started

```
// before: one global map, alive for the whole app session
static final _routers = <String, GoRouter>{};

Widget checkoutPage() {
  final router = _routers.putIfAbsent('/checkout', buildRouter);
  return Router.withConfig(config: router); // reused on every visit
}
```

BUG Login → Back → Login: no button worked. The new visit got the old, broken router.

BUG A page animation kept two pages alive → one router shown twice → duplicate GlobalKey.

BUG The router stayed on "too many attempts" → coming back looped until the app was killed.

We patched each bug: 9 places now clear the cache by hand. Nobody calls dispose().

The router lives and dies with its host page

packages/common/routing - EmbeddedRouter: written once

```dart
class _EmbeddedRouterState extends State<EmbeddedRouter> {
  late final _composition = widget.create();            // created with the page
  @override
  Widget build(BuildContext context) =>
      Router.withConfig(config: _composition.router);
  @override
  void dispose() { _composition.dispose(); super.dispose(); }   // disposed with it
}
```

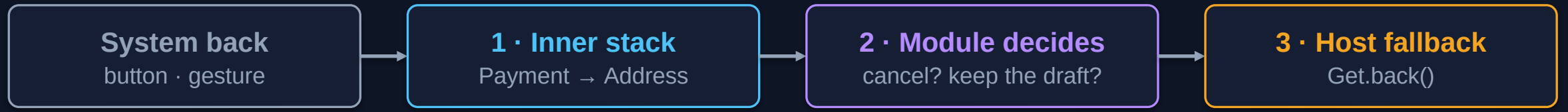
lib/app - the GetX page for /checkout just uses it

```dart
EmbeddedRouter(create: () => RouteComposition([checkout.routes]))
```



Created together. Disposed together. One router per mounted module - not per screen.

Back: the module decides first

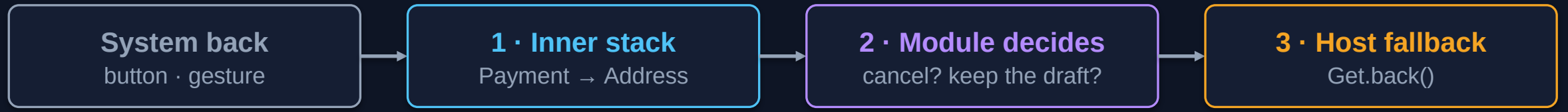


lib/app - the GetX page for /checkout

```
PopScope(                                          // the GetX page: no State needed
  canPop: false,                                   // GetX never pops it alone
  onPopInvokedWithResult: (didPop, _) {
    if (didPop) return;
    if (checkout.requestBack() == BackResolution.handled) return;  // 1 + 2
    Get.back();                                    // 3: host fallback
  },
  child: EmbeddedRouter(create: …),
)
```

BUG a PIN reset flow used go() → one page in the inner stack → Back closed the whole flow.

Back: the module decides first



iOS

canPop: false turns off the edge swipe at the module root.

→ The visible Back button is the only way out.

Android

System Back and the back gesture go through the same chain.

→ With canPop: false, no predictive-back animation.

Web

Today two routers write the same address bar: the old one and the embedded one.

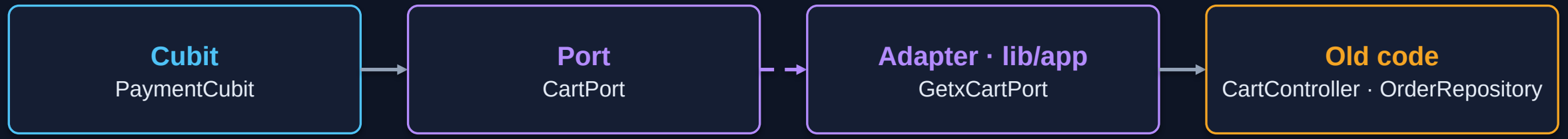
→ Target: one router owns the URL (compose).

One chain for every Back in the app: system button · back gesture · the arrow on the screen.

Port 1 · data: the old code keeps working

packages/modules/checkout - the module owns the port, in its own types

```dart
abstract interface class CartPort {
  Future<Cart> currentCart();                    // the old app owns the cart
  Future<OrderId> placeOrder(OrderDraft draft);  // the old, tested order API
}
```



EXAMPLE Change PIN: new screens, old PIN API. Port: verifyOldPin() · setNewPin(). HTTP and the token stay in the old app.

New UI, old code, a small port between them.

Port 2 · exit: what happened, not where to go

module - WHAT happened

```dart
abstract interface class CheckoutHostPort {
  Future<void> orderPlaced(OrderId id);
  Future<void> cancelled();
  Future<void> signInRrequired();
}
```

Later · go_router shell

cancelled() → **router.go('/cart')**

The module does not change.

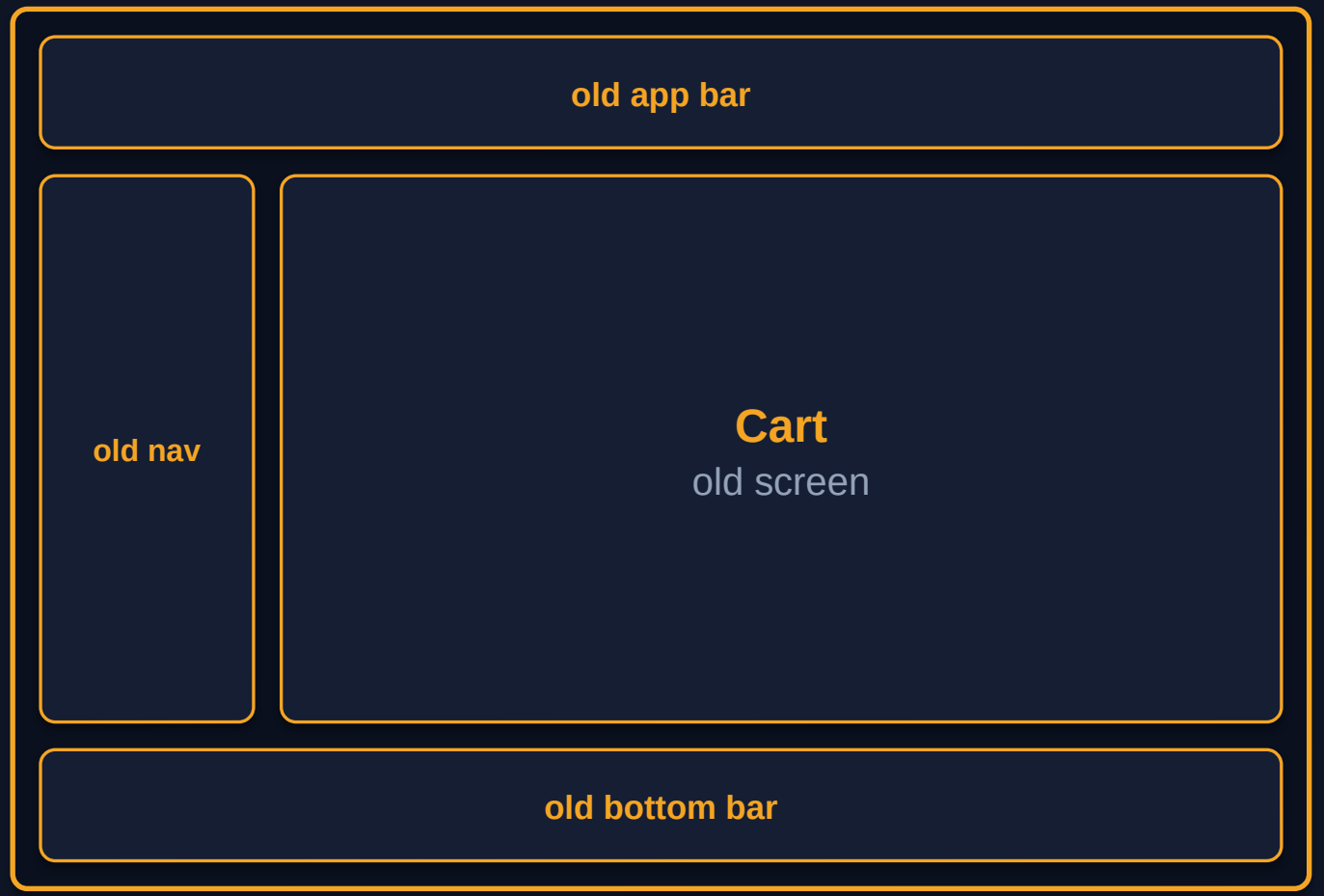
lib/app/adapters - the host decides WHERE to go

```dart
final class GetxCheckoutHost implements CheckoutHostPort {
  Future<void> orderPlaced(OrderId id) async => Get.offNamed('/orders/placed');
  Future<void> cancelled() async => Get.back();
  Future<void> signInRequired() async => Get.offAllNamed('/login');
}
```

a11y: the module is gone after the exit → the host announces the result, focus starts on the next screen.

EXAMPLE Change PIN reports 1 of 4 outcomes. Settings (old code) decides what's next and announces success.

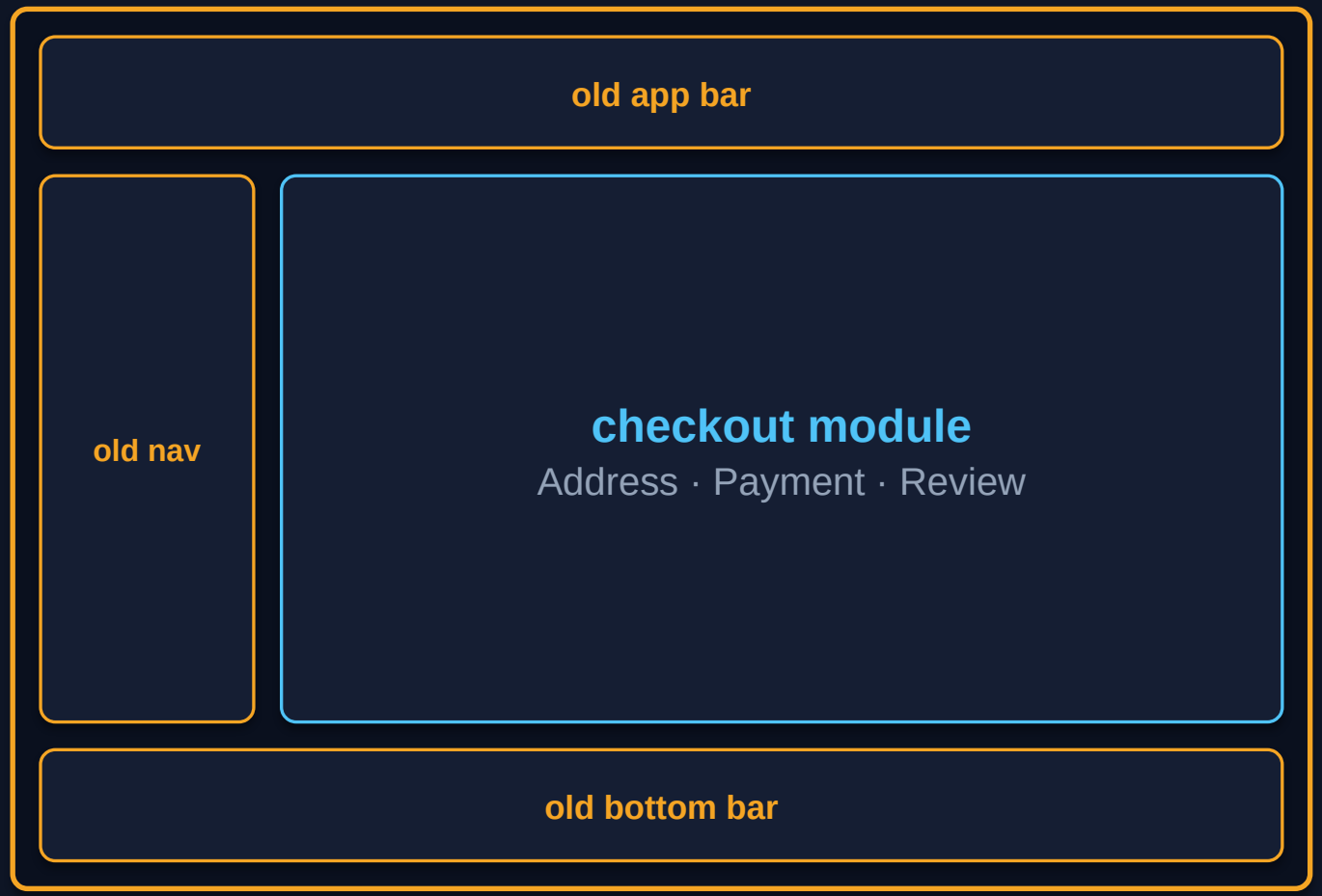
Port 3 · frame: swap the frame, keep the module



STEP 1 · old

Old frame, old screen.

Port 3 · frame: swap the frame, keep the module



STEP 2 · mixed

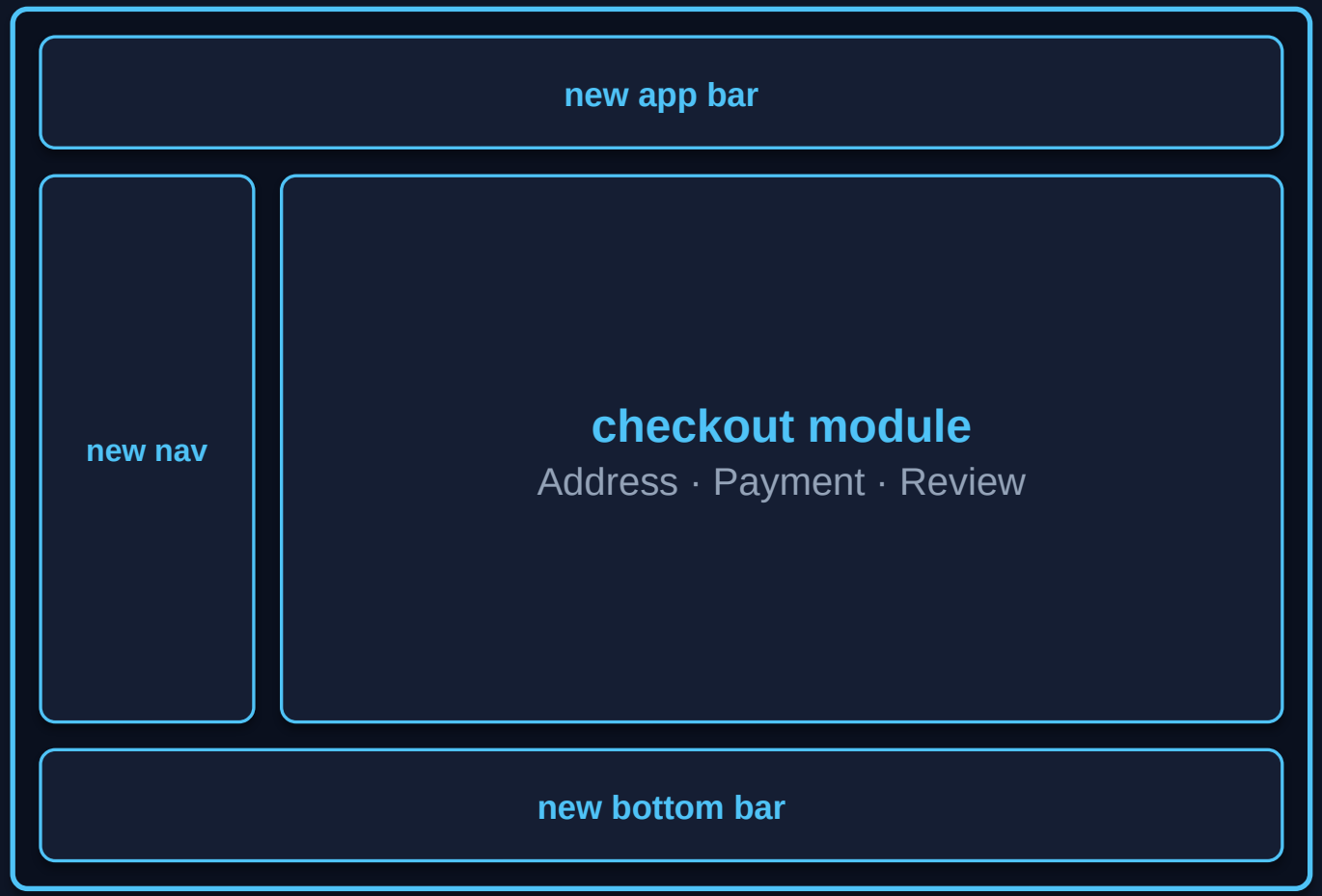
New module inside the old frame.

lib/app

```
CheckoutModule.create(
  host: GetxCheckoutHost(),
  chrome: ModuleChrome(
    appBar: LegacyAppBar.new,
    sideNav: LegacyRail.new,
    bottomNav: LegacyBottomBar.new,
  ),
)
```

The module never imports the old widgets. The frame keeps its nav landmark for screen readers.

Port 3 · frame: swap the frame, keep the module



STEP 3 · target

New go_router shell: new frame, same module.

new shell

```
StatefulShellRoute.indexedStack(
  builder: (_, __, shell) =>
      NewFrame(child: shell),
  branches: [cart, checkout],
)
```

The module did not change. Only the frame adapter did.

A test guards the boundary

test/architecture/ - inside the module package

```
test('no file imports the old app', () {
  expectNoImport(allFiles(), 'package:my_app/');
});
test('no file reaches for GetX', () {
  expectNoImport(allFiles(), 'package:get/');
});
test('no file imports another module', () {
  expectNoImport(allFiles(), siblingModules());
});
```

A module may import

- ✓ core · network
- ✓ routing · design_system

Never

- ✕ package:get
  GetX
- ✕ package:my_app
  the old app
- ✕ package:auth
  another module

Plain Dart: read the files, check the imports. **Every adapter has a delete condition.**

Where old meets new, accessibility breaks quietly

Most a11y bugs are normal bugs. These four come from mixing old and new.

WHEN	WHAT CAN BREAK BETWEEN OLD AND NEW	WHAT WE DO
Open a screen	The old app says "page loaded" from a GetX observer. It can't see pages inside the module's router.	The module's template announces each page. Now shared in the design system.
Type	BUG Our new floating nav bar rebuilt the old screens when the keyboard opened. The field lost focus.	Fixed once, in the frame. Every screen inside got the fix.
Go back	iOS: no swipe at the module root. The Back button is the only way out.	The template always shows Back. Test it with big text and a keyboard.
Get the result	After the exit, the module is gone.	The host announces the result (port 2).

One fix in a shared widget reaches every screen that uses it. But not the copies.

One source of truth for both hosts

packages/common/design_system - tokens: one theme for both hosts

```dart
final appTheme = ThemeData(extensions: const [AppTokens.light]);
GetMaterialApp(theme: appTheme)       // old host
MaterialApp.router(theme: appTheme)   // new host
```

template owns: safe area · keyboard · scroll · sticky button · visible Back

```dart
CheckoutStepTemplate(                 // the screen gives content only
  title: context.l10n.addressTitle,
  body: const AddressForm(),
  primaryAction: PrimaryButton(
    label: context.l10n.continueLabel,
    onPressed: context.read<AddressCubit>().submit,
  ),
)
```

TODAY both hosts use the same theme object · our page template keeps the focused field above the keyboard.

TODAY our auth module still has 31 copied design-system files, with its own focus ring and Back link. Shared fixes don't reach copies.

Accessibility as a test, at the seams

test/ - a new screen, tested where old meets new

```
testWidgets('address step in the old frame at 2× text', (tester) async {
  await tester.pumpWidget(LegacyFrameApp(              // old theme + app bar + nav
    child: MediaQuery(
      data: const MediaQueryData(
          size: Size(390, 844), textScaler: TextScaler.linear(2.0)),
      child: const AddressStep(),                      // new module screen
    ),
  ));
  await expectLater(tester, meetsGuideline(textContrastGuideline));
  await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
});
```

EVERY PR

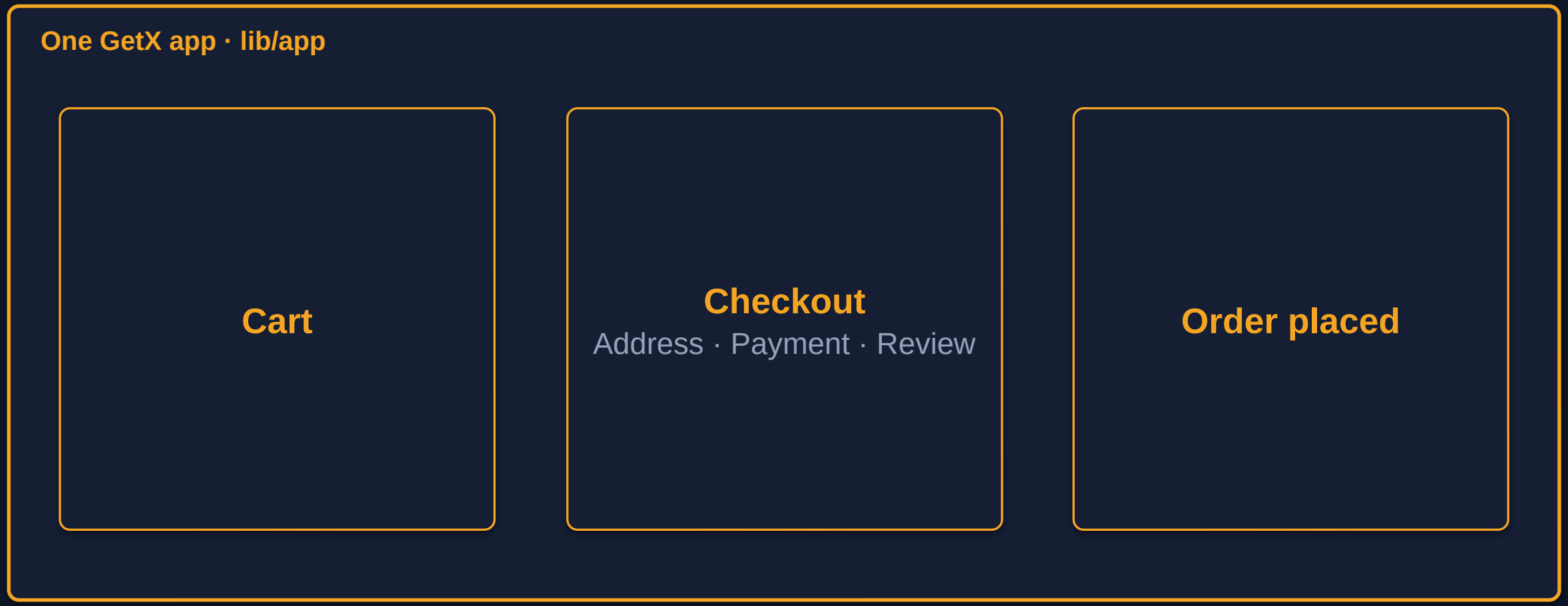
- guidelines at 2× text
- semantics snapshot
- focus survives the keyboard

EVERY RELEASE

TalkBack · VoiceOver

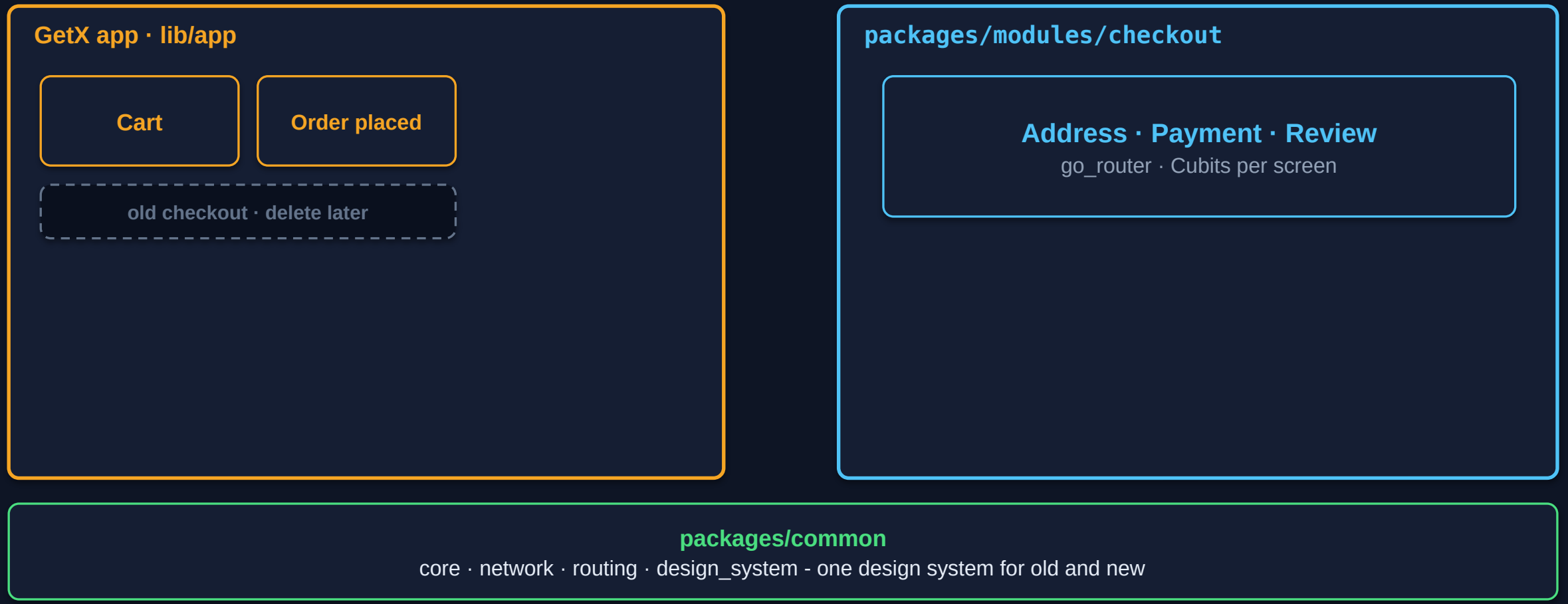
TODAY guideline checks in ~50 test files · design-system widgets tested at up to 4× text · an a11y test-helpers package.

The whole bridge, step by step



One big GetX app. Checkout is the next part to rewrite.

The whole bridge, step by step



Write the journey again, in a module. The old one stays - for now.

The whole bridge, step by step

GetX app · lib/app

Cart

Order placed

old checkout · delete later

packages/modules/checkout

handle

ports

Address · Payment · Review

go_router · Cubits per screen

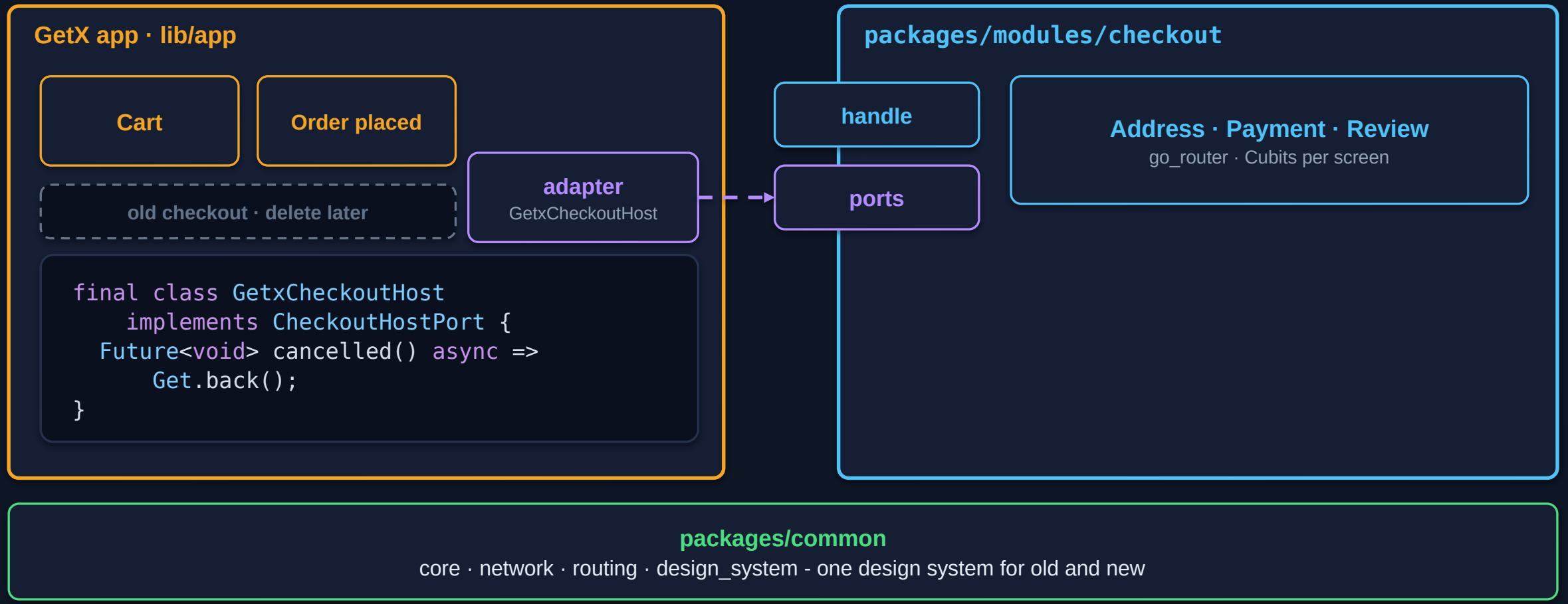
```
abstract interface class
    CheckoutHostPort {
  Future<void> orderPlaced(OrderId id);
  Future<void> cancelled();
}
```

packages/common

core · network · routing · design_system - one design system for old and new

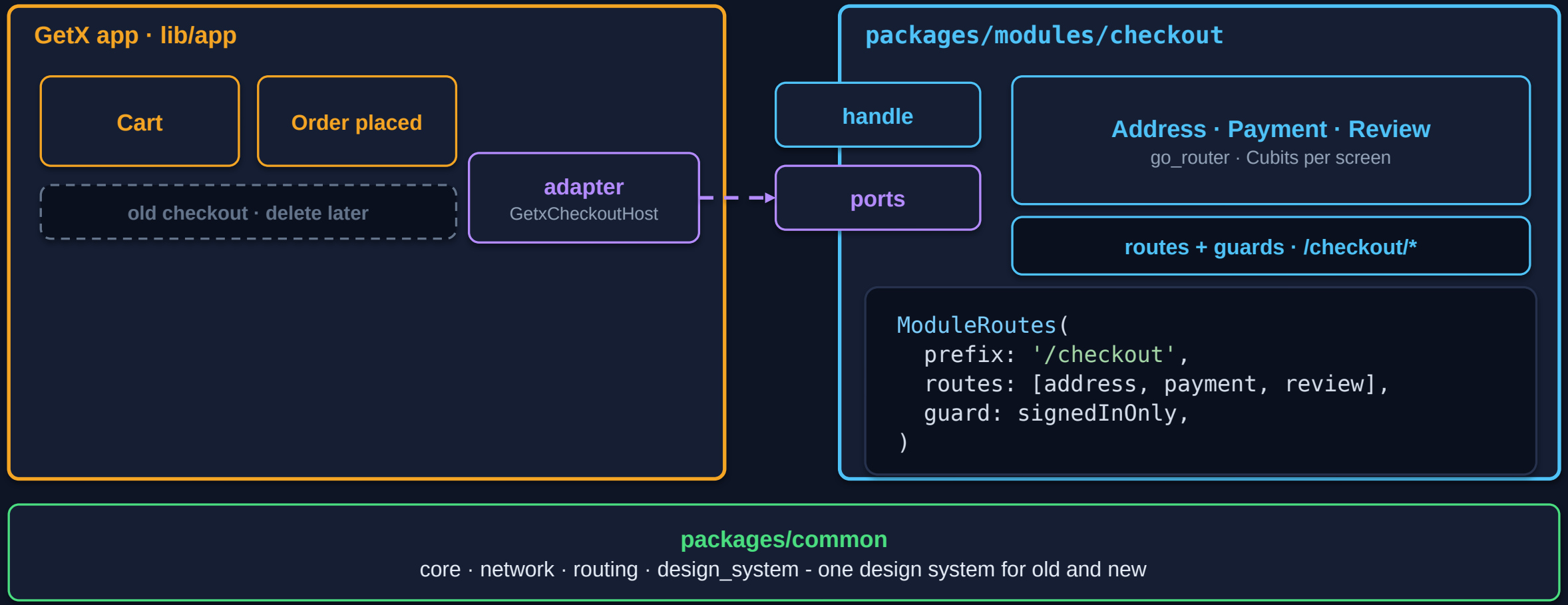
The module owns its ports: the handle for the host, and what it needs.

The whole bridge, step by step



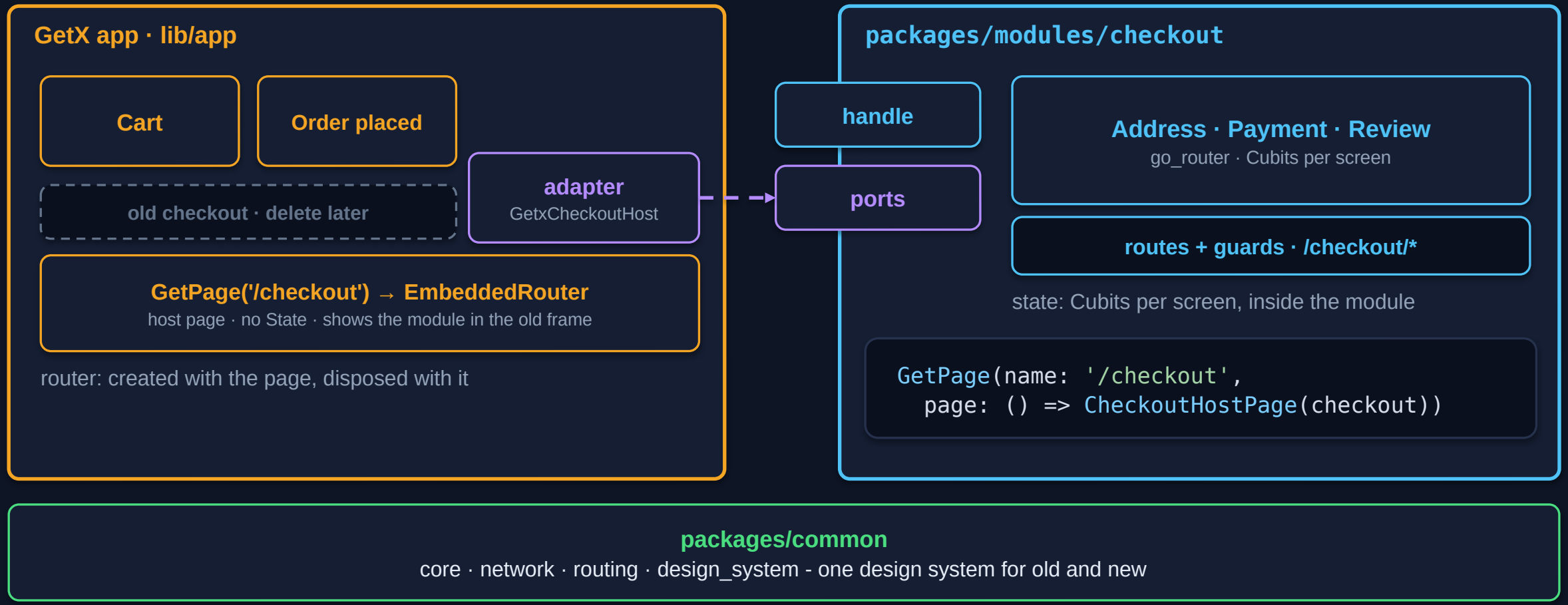
The host implements the port with old code. Imports point in, calls go out.

The whole bridge, step by step



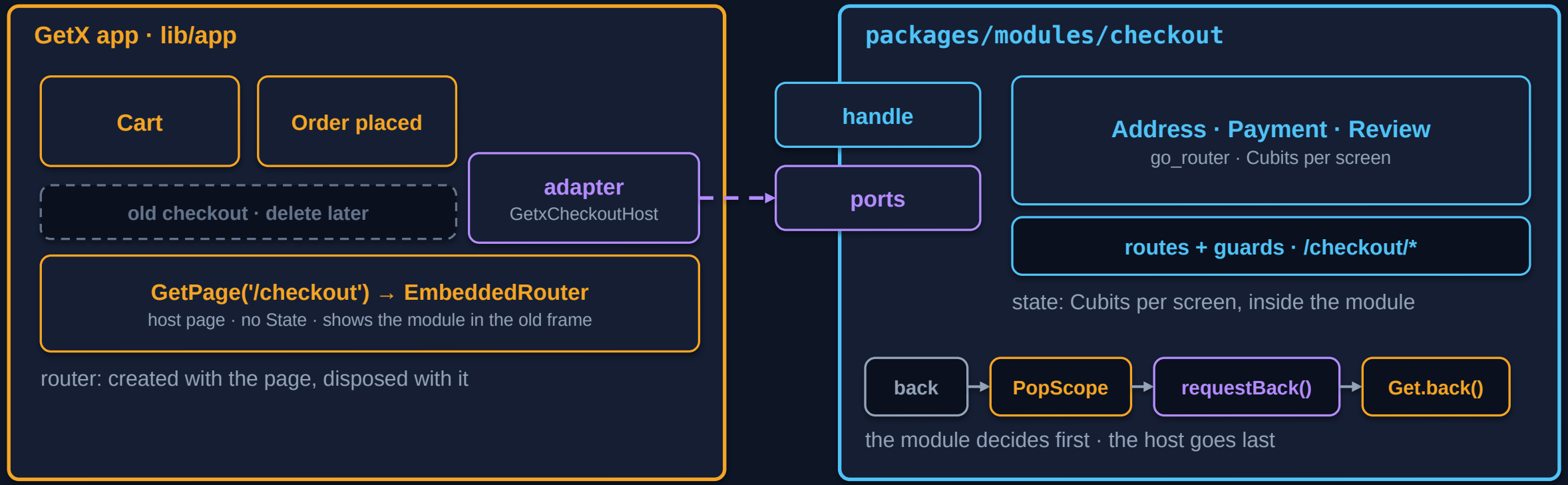
The module gives its routes and their guards together - built on go_router.

The whole bridge, step by step



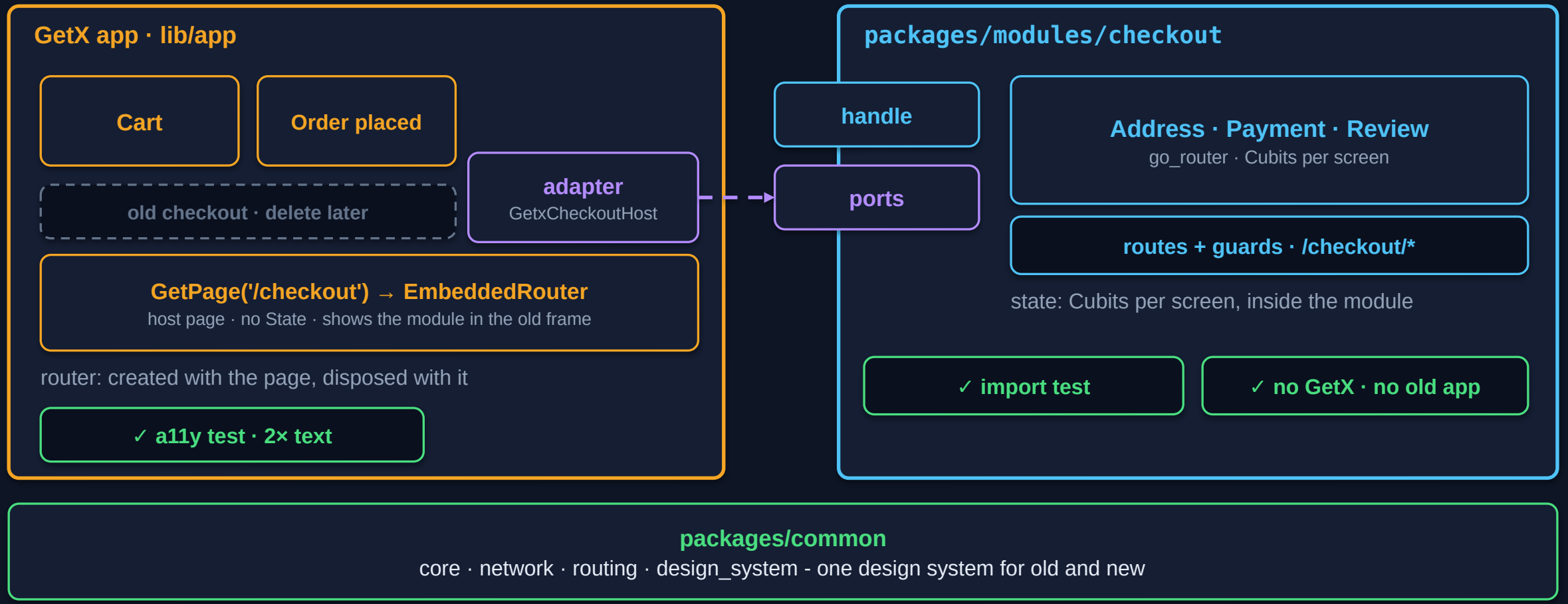
The GetX page shows the module. EmbeddedRouter owns the router; the module owns the state.

The whole bridge, step by step



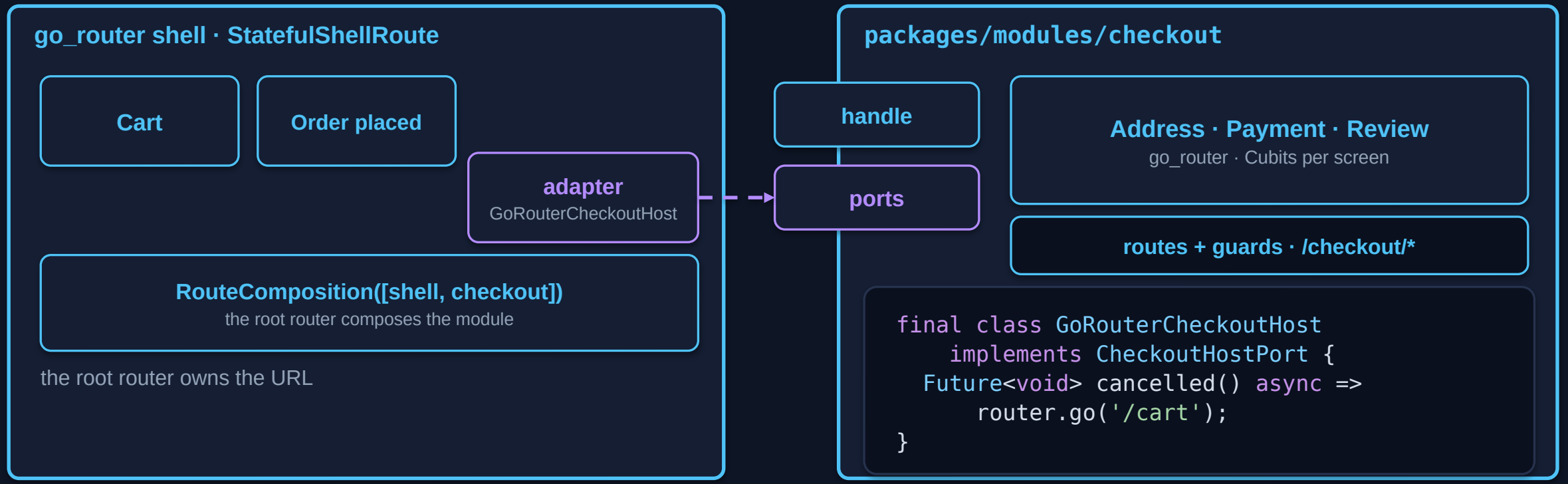
When the user presses Back, the module decides first. The host goes last.

The whole bridge, step by step



Tests keep the rules: an import test and accessibility tests.

The whole bridge, step by step



Swap the host and the adapter. The module stays. The old checkout is gone - strangler fig.

Take home

1

One owner per decision.

EmbeddedRouter owns the router · the module owns Back · the host owns where to go.

2

Ports in the module, adapters in the host.

Old code and GetX stay in the host - a small test keeps them out of modules.

3

Fix the seams, not only the screens.

One design system for old and new · test a11y where old meets new.



Message me on LinkedIn for AI skills

START TOMORROW

freeze legacy · design system first · one module behind ports · embed · swap adapters · delete the bridge

Thank you!

Mateusz Rus · let's talk after the session!